PRACTICAL - 09

Part A — Practice Programs (Hour 1)

PROBLEM1	1. Counter race: several threads each increment a shared counter many times. Run it without synchronization to watch the total come out too low, then make the increment synchronized and confirm it is exactly right every run.
CODE	

```
class Counter {
   int count = 0;

   void increment() {
      count++;
   }
}

class CounterThread extends Thread {
   Counter counter;
   int times;

   CounterThread(Counter counter, int times) {
      this.counter = counter;
      this.times = times;
   }

   public void run() {
      for (int i = 0; i < times; i++) {
         counter.increment();
      }
   }
}

public class CounterRace {

   public static void main(String[] args) throws Exception {

      int threadCount = 10;
      int times = 100000;

      // Without synchronization
      Counter counter = new Counter();
      Thread[] threads = new Thread[threadCount];

      for (int i = 0; i < threadCount; i++) {
         threads[i] = new CounterThread(counter, times);
         threads[i].start();
      }
```

```
        for (Thread thread : threads) {
            thread.join();
        }

        System.out.println("Without synchronization: " + counter.count);
        System.out.println("Expected: " + (threadCount * times));

        // With synchronization
        Counter counter2 = new Counter();
        Thread[] threads2 = new Thread[threadCount];

        for (int i = 0; i < threadCount; i++) {

            threads2[i] = new Thread(() -> {
                for (int j = 0; j < times; j++) {

                    synchronized (counter2) {
                        counter2.count++;
                    }

                }
            });

            threads2[i].start();
        }

        for (Thread thread : threads2) {
            thread.join();
        }

        System.out.println("With synchronization: " + counter2.count);
        System.out.println("Expected: " + (threadCount * times));
    }
}
```

OUTPUT

```
PS C:\Users\aimlo\OneDrive\Desktop\CEUE203 OOP\PRACTICAL 9> javac CounterRace.java
PS C:\Users\aimlo\OneDrive\Desktop\CEUE203 OOP\PRACTICAL 9> java CounterRace
Without synchronization: 191517
Expected: 1000000
With synchronization: 1000000
Expected: 1000000
```

PROBLEM2	2. Seat booking race: a shared seatsLeft of 5 with about 10 threads each trying to book one seat. Without synchronization show that more than 5 can succeed (oversell); with a synchronized book() exactly 5 do.
CODE	

```
class SeatBooking {
   int seatsLeft = 5;

   void book() {
      if (seatsLeft > 0) {
         try {
            Thread.sleep(10);
         } catch (InterruptedException e) {
            e.printStackTrace();
         }

         seatsLeft--;
         System.out.println(Thread.currentThread().getName()
               + " booked a seat.");
      } else {
         System.out.println(Thread.currentThread().getName()
               + " failed - No seats left.");
      }
   }

   synchronized void bookSynchronized() {
      if (seatsLeft > 0) {
         try {
            Thread.sleep(10);
         } catch (InterruptedException e) {
            e.printStackTrace();
         }

         seatsLeft--;
         System.out.println(Thread.currentThread().getName()
               + " booked a seat.");
      } else {
         System.out.println(Thread.currentThread().getName()
               + " failed - No seats left.");
      }
   }
}

public class SeatBookingRace {

   public static void main(String[] args) throws Exception {

      // Without synchronization
      SeatBooking booking = new SeatBooking();
```

```
        Thread[] threads = new Thread[10];

        System.out.println("WITHOUT SYNCHRONIZATION:");

        for (int i = 0; i < 10; i++) {
            threads[i] = new Thread(
                    () -> booking.book(),
                    "Thread-" + (i + 1)
            );
            threads[i].start();
        }

        for (Thread thread : threads) {
            thread.join();
        }

        System.out.println("Seats left: " + booking.seatsLeft);

        // With synchronization
        SeatBooking booking2 = new SeatBooking();

        Thread[] threads2 = new Thread[10];

        System.out.println("\nWITH SYNCHRONIZATION:");

        for (int i = 0; i < 10; i++) {
            threads2[i] = new Thread(
                    () -> booking2.bookSynchronized(),
                    "Thread-" + (i + 1)
            );
            threads2[i].start();
        }

        for (Thread thread : threads2) {
            thread.join();
        }

        System.out.println("Seats left: " + booking2.seatsLeft);
    }
}
```

OUTPUT

```
PS C:\Users\aimlo\OneDrive\Desktop\CEUE203 OOP\PRACTICAL 9> javac .\SeatBookingRace.java
PS C:\Users\aimlo\OneDrive\Desktop\CEUE203 OOP\PRACTICAL 9> java SeatBookingRace
WITHOUT SYNCHRONIZATION:
Thread-1 booked a seat.
Thread-5 booked a seat.
Thread-6 booked a seat.
Thread-9 booked a seat.
Thread-8 booked a seat.
Thread-2 booked a seat.
Thread-7 booked a seat.
Thread-4 booked a seat.
Thread-10 booked a seat.
Thread-3 booked a seat.
Seats left: -5

WITH SYNCHRONIZATION:
Thread-1 booked a seat.
Thread-2 booked a seat.
Thread-10 booked a seat.
Thread-4 booked a seat.
Thread-6 booked a seat.
Thread-5 failed - No seats left.
Thread-7 failed - No seats left.
Thread-8 failed - No seats left.
Thread-9 failed - No seats left.
Thread-3 failed - No seats left.
Seats left: 0
```

PROBLEM3

3. (For Advanced Learners – home) Split a large array of numbers across several threads that add into one shared total, show the wrong result, then fix it two ways and compare; decide how to measure the difference.

CODE

```
import java.util.concurrent.atomic.AtomicLong;

class SharedTotal {
   long total = 0;

   void add(long value) {
      total += value;
   }

   synchronized void addSynchronized(long value) {
      total += value;
   }
}

public class ArraySumRace {

   static int[] arr = new int[1_000_000];
   static int threadCount = 10;

   public static void main(String[] args) throws Exception {

      // Fill array with 1
```

```
for (int i = 0; i < arr.length; i++) {
   arr[i] = 1;
}

System.out.println("Expected Total: " + arr.length);

// 1. Without Synchronization
SharedTotal total1 = new SharedTotal();

long start1 = System.nanoTime();

runThreads((start, end) -> {
   for (int i = start; i < end; i++) {
      total1.add(arr[i]);
   }
});

long end1 = System.nanoTime();

System.out.println("\nWithout Synchronization:");
System.out.println("Total = " + total1.total);
System.out.println("Time = " + (end1 - start1) / 1_000_000.0 + " ms");


// 2. With Synchronization
SharedTotal total2 = new SharedTotal();

long start2 = System.nanoTime();

runThreads((start, end) -> {
   for (int i = start; i < end; i++) {
      total2.addSynchronized(arr[i]);
   }
});

long end2 = System.nanoTime();

System.out.println("\nWith Synchronization:");
System.out.println("Total = " + total2.total);
System.out.println("Time = " + (end2 - start2) / 1_000_000.0 + " ms");


// 3. Using AtomicLong
AtomicLong total3 = new AtomicLong(0);

long start3 = System.nanoTime();
```

```
        runThreads((start, end) -> {
            for (int i = start; i < end; i++) {
                total3.addAndGet(arr[i]);
            }
        });

        long end3 = System.nanoTime();

        System.out.println("\nUsing AtomicLong:");
        System.out.println("Total = " + total3.get());
        System.out.println("Time = " + (end3 - start3) / 1_000_000.0 + " ms");
    }


    interface Task {
        void execute(int start, int end);
    }


    static void runThreads(Task task) throws Exception {

        Thread[] threads = new Thread[threadCount];

        int partSize = arr.length / threadCount;

        for (int i = 0; i < threadCount; i++) {

            int start = i * partSize;

            int end = (i == threadCount - 1)
                    ? arr.length
                    : start + partSize;

            threads[i] = new Thread(() -> task.execute(start, end));

            threads[i].start();
        }

        for (Thread thread : threads) {
            thread.join();
        }
    }
}
```

OUTPUT

```
PS C:\Users\aimlo\OneDrive\Desktop\CEUE203 OOP> cd "PRACTICAL 9"
>> javac ArraySumRace.java
>> java ArraySumRace
Expected Total: 1000000

Without Synchronization:
Total = 122186
Time = 54.73 ms

With Synchronization:
Total = 1000000
Time = 104.2852 ms

Using AtomicLong:
Total = 1000000
Time = 42.2305 ms
PS C:\Users\aimlo\OneDrive\Desktop\CEUE203 OOP\PRACTICAL 9>
```